

Homework 03

Question 1

See explain-to-me skill at end of document.

Question 2

a.

```
set.seed(43231)
n <- 100
L <- rnorm(n); U1 <- rnorm(n); U2 <- rnorm(n)
X3 <- rnorm(n); X4 <- rnorm(n)
X1 <- L + 0.04*U1; X2 <- L + 0.04*U2
Xraw <- cbind(X1, X2, X3, X4)
standardize <- function(x) {
  xbar <- colMeans(x)
  s <- sqrt(colMeans((sweep(Xraw, 2, xbar))^2))
  sweep(sweep(Xraw, 2, xbar), 2, s, "/")
}
X <- standardize(Xraw); p <- ncol(X)
corr12 <- cor(X[, 1], X[, 2]); svals <- svd(X)$d
cat("corr =", round(corr12,4), " svals =", round(svals,4), "\n")
```

```
corr = 0.9977  svals = 14.1415 10.3477 9.629 0.4754
```

X_1, X_2 share latent L plus small noise, so they're nearly identical after standardizing; $d_{\min} = 0.4754 \approx 30\times$ smaller than $d_{\max} = 14.1415$. Since $\text{Var}(\hat{\beta}) \propto \mathbf{D}^{-2}$, $\hat{\beta}_1 - \hat{\beta}_2$ aligns with $d_{\min}$ (variance factor ≈ 4.4) while $\hat{\beta}_1 + \hat{\beta}_2$ aligns with $d_{\max}$ (factor ≈ 0.005) — a $\approx 900\times$ gap.

b.

```
beta_true <- c(1.5, 1.5, 1, 0)
mu <- as.numeric(X %*% beta_true)
lambdas <- c(0, 0.02, 0.2); nrep <- 200
XtX <- crossprod(X); I_p <- diag(p)
out <- setNames(vector("list", 3), as.character(lambdas))
for (lam in names(out)) out[[lam]] <- list(b1=numeric(nrep), b2=numeric(nrep),
  sum=numeric(nrep), diff=numeric(nrep), tr=numeric(nrep), te=numeric(nrep))
for (r in 1:nrep) {
  eps <- rnorm(n); eps_ <- rnorm(n)
  y <- mu + eps; ystar <- mu + eps_
  ybar <- mean(y); ytil <- y - ybar; Xty <- crossprod(X, ytil)
  for (lam in lambdas) {
    key <- as.character(lam)
    beta <- solve(XtX + n*lam*I_p, Xty)
```

```

    fitted <- ybar + as.numeric(X %*% beta)
    out[[key]]$b1[r] <- beta[1]; out[[key]]$b2[r] <- beta[2]
    out[[key]]$sum[r] <- beta[1]+beta[2]; out[[key]]$diff[r] <- beta[1]-beta[2]
    out[[key]]$tr[r] <- mean((y-fitted)^2); out[[key]]$te[r] <- mean((ystar-fitted)^2)
  }
}
summary_tab <- do.call(rbind, lapply(names(out), function(key) {
  d <- out[[key]]
  data.frame(lambda=as.numeric(key), mean_b1=mean(d$b1), sd_b1=sd(d$b1),
    mean_b2=mean(d$b2), sd_b2=sd(d$b2), mean_sum=mean(d$sum), sd_sum=sd(d$sum),
    mean_diff=mean(d$diff), sd_diff=sd(d$diff), train_MSE=mean(d$tr), test_MSE=mean(d$te))
}))
print(round(summary_tab, 4), row.names = FALSE)

```

```

lambda mean_b1 sd_b1 mean_b2 sd_b2 mean_sum sd_sum mean_diff sd_diff
0.00  1.4956 1.5439  1.5098 1.5486   3.0054 0.0951  -0.0142  3.0911
0.02  1.4870 0.1615  1.4892 0.1659   2.9762 0.0942  -0.0022  0.3136
0.20  1.3679 0.0457  1.3687 0.0473   2.7366 0.0865  -0.0008  0.0344
train_MSE test_MSE
0.9648    1.0625
0.9747    1.0493
1.0768    1.1335

```

c.

Under OLS, $\hat{\beta}_1, \hat{\beta}_2$ have $\text{sd} \approx 1.55$, larger than the true value 1.5. Ridge fixes this: at $\lambda = 0.02$ sd drops $\approx 10\times$ with almost no bias; at $\lambda = 0.2$ sd shrinks further but means drop $\approx 9\%$ (shrinkage bias). The sum is stable under OLS ($\text{sd} = 0.095$) and stays near-unbiased at $\lambda = 0.02$; the difference collapses from $\text{sd} \approx 3.09$ to 0.31 to 0.03 while its mean stays near 0 — ridge removes variance, not bias, in that direction. Test MSE improves slightly from $\lambda = 0 \rightarrow 0.02$ ($1.0625 \rightarrow 1.0493$) then worsens at $\lambda = 0.2$ (1.1335) — the usual bias-variance U-shape. Since $x_{i1} \approx x_{i2}$, fitted values depend almost entirely on the well-identified sum, not the split, so ridge's huge variance cut on individual coefficients doesn't produce an equally large change in predictions.

Question 3

a.

$$(\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I}_p) \hat{\beta}_\lambda = \mathbf{X}^T \tilde{\mathbf{y}}$$

from setting $\nabla L_\lambda = 0$. $\mathbf{X}^T \mathbf{X}$ is PSD; adding $n\lambda \mathbf{I}_p$ ($\lambda > 0$) makes it positive definite, hence invertible, regardless of $\text{rank}(\mathbf{X})$ or $p > n$ — so $\hat{\beta}_\lambda$ always exists uniquely.

b.

With $\mathbf{X} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^T$, $\mathbf{X} \hat{\beta}_\lambda = \sum_{j=1}^r \rho_j(\lambda) \mathbf{u}_j (\mathbf{u}_j^T \tilde{\mathbf{y}})$, $\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda)$.

```

n <- 100; lambda <- 0.01; d <- c(10, 2, 0.2)
rho <- d^2 / (d^2 + n*lambda); df_eff <- 1 + sum(rho)
cat("rho:", round(rho,4), " df_eff:", round(df_eff,4), "\n")

```

```
rho: 0.9901 0.8 0.0385 df_eff: 2.8286
```

$\rho_1 \approx 0.9901$, $\rho_2 = 0.8$, $\rho_3 \approx 0.0385$; $\text{df}_{\text{eff}} \approx 2.829$.

c.

$d_3 = 0.2$ receives the strongest shrinkage ($\rho_3 \approx 0.038$). OLS variance $\propto 1/d_j^2$, so shrinking a small- d_j direction removes mostly noise, reducing variance. But if the true β has a real component along that direction, shrinking it removes signal too — producing substantial bias if a weak/collinear direction is actually predictive.

Question 4

a.

```
eig_A0 <- c(9, 0.01); lambda <- 0.09; eig_Alam <- eig_A0 + lambda
cond_A0 <- max(eig_A0)/min(eig_A0); cond_Alam <- max(eig_Alam)/min(eig_Alam)
cat("cond(A0):", cond_A0, " cond(Alam):", round(cond_Alam,4), "\n")
```

```
cond(A0): 900  cond(Alam): 90.9
```

$\kappa(\mathbf{A}_0) = 900$, $\kappa(\mathbf{A}_\lambda) = 90.9$. Adding $\lambda \mathbf{I}_p$ inflates the small eigenvalue proportionally more than the large one, turning elongated level sets closer to circular — ridge improves conditioning, not just the estimate.

b.

$\mathbf{e}^{(k+1)} = (\mathbf{I} - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}$, so along an eigendirection $e_j^{(k+1)} = (1 - \eta(a + \lambda))e_j^{(k)}$.

```
M <- 9.09; a_vals <- c(9, 0.01); lambda <- 0.09
cat("interval: (0,", round(2/M,4), ")\n")
```

```
interval: (0, 0.22 )
```

```
eta1 <- 1/M; cat("eta=1/M factors:", round(1 - eta1*(a_vals+lambda),4), "\n")
```

```
eta=1/M factors: 0 0.989
```

```
eta2 <- 2.1/M; cat("eta=2.1/M factors:", round(1 - eta2*(a_vals+lambda),4), "\n")
```

```
eta=2.1/M factors: -1.1 0.9769
```

Contracting interval: $0 < \eta < 2/M \approx 0.22$. At $\eta = 1/M$: factors 0 and ≈ 0.989 — the M -direction converges in one step, the small-eigenvalue direction much slower. At $\eta = 2.1/M$: M -direction factor = -1.1 , $|-1.1| > 1 \Rightarrow$ diverges.

c.

Convergence speed is an optimization property, not a predictive one. Maximizing it would push $\lambda \rightarrow \infty$ (best conditioning), forcing $\hat{\beta}_\lambda \rightarrow \mathbf{0}$ — heavily biased. Valid tuning (e.g. CV) minimizes expected prediction error, unrelated to solver speed.

Question 5

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
data <- read.csv("data/realestate.csv")
folds <- read.csv("data/realestate-split-folds.csv")
df <- merge(data, folds, by = "row_id")
pred_cols <- c("date", "age", "distance", "stores", "latitude", "longitude")
train <- df[df$split == "train", ]; test <- df[df$split == "test", ]
Xtrain_raw <- as.matrix(train[, pred_cols]); ytrain <- train$price; cvfold <- train$cv_fold
Xtest_raw <- as.matrix(test[, pred_cols]); ytest <- test$price
p <- ncol(Xtrain_raw); lam_grid <- c(0, 10^(-4 + 0.05*(0:120)))

standardize_fit <- function(Xraw) {
  xbar <- colMeans(Xraw); s <- sqrt(colMeans(sweep(Xraw, 2, xbar)^2))
  list(xbar = xbar, s = s)
}
apply_standardize <- function(Xraw, par) sweep(sweep(Xraw, 2, par$xbar), 2, par$s, "/")
ridge_fit <- function(X, y, lam) {
  n <- nrow(X); p <- ncol(X); ybar <- mean(y); ytil <- y - ybar
  beta <- solve(crossprod(X) + n*lam*diag(p), crossprod(X, ytil))
  list(beta0 = ybar, beta = beta)
}
```

a.

```
nfold <- 10
fold_errs <- matrix(NA, nrow = nfold, ncol = length(lam_grid))
for (k in 1:nfold) {
  val_idx <- cvfold == k; tr_idx <- !val_idx
  par <- standardize_fit(Xtrain_raw[tr_idx, , drop = FALSE])
  Xtr <- apply_standardize(Xtrain_raw[tr_idx, , drop = FALSE], par)
  Xval <- apply_standardize(Xtrain_raw[val_idx, , drop = FALSE], par)
  ytr <- ytrain[tr_idx]; yval <- ytrain[val_idx]
  for (li in seq_along(lam_grid)) {
    fit <- ridge_fit(Xtr, ytr, lam_grid[li])
    pred <- fit$beta0 + Xval %*% fit$beta
    fold_errs[k, li] <- mean((yval - pred)^2)
  }
}
mean_mse <- colMeans(fold_errs); se_mse <- apply(fold_errs, 2, sd) / sqrt(nfold)
imin <- which.min(mean_mse); lambda_min <- lam_grid[imin]
threshold <- mean_mse[imin] + se_mse[imin]
lambda_1se <- max(lam_grid[mean_mse <= threshold])
cat("lambda_min =", lambda_min, " lambda_1se =", lambda_1se, "\n")
```

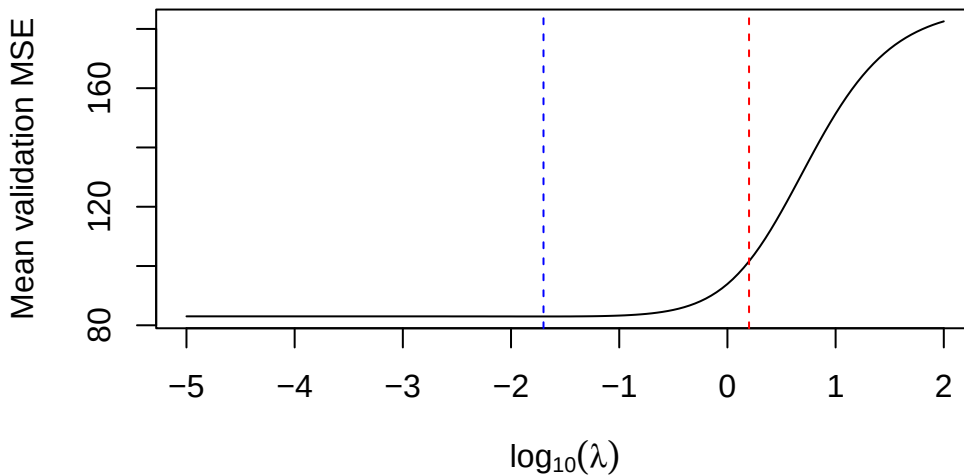
lambda_min = 0.01995262 lambda_1se = 1.584893

```

plot_lam <- ifelse(lam_grid == 0, min(lam_grid[lam_grid>0])/10, lam_grid)
plot(log10(plot_lam), mean_mse, type = "l", xlab = expression(log[10](lambda)),
     ylab = "Mean validation MSE", main = "10-fold CV curve")
abline(v = log10(lambda_min), col = "blue", lty = 2)
abline(v = log10(lambda_1se), col = "red", lty = 2)

```

10-fold CV curve



$\lambda_{\min} \approx 0.0200$ (mean CV MSE ≈ 83.0 , SE ≈ 19.2); $\lambda_{1se} \approx 1.585$.

b.

```

par_all <- standardize_fit(Xtrain_raw)
Xall <- apply_standardize(Xtrain_raw, par_all); n <- nrow(Xall)
dvals <- svd(Xall)$d
gcv_vals <- numeric(length(lam_grid)); df_effs <- numeric(length(lam_grid))
for (li in seq_along(lam_grid)) {
  lam <- lam_grid[li]
  fit <- ridge_fit(Xall, ytrain, lam)
  fitted <- fit$beta0 + Xall %*% fit$beta
  train_mse <- mean((ytrain - fitted)^2)
  rho <- dvals^2 / (dvals^2 + n*lam); df_eff <- 1 + sum(rho)
  gcv_vals[li] <- train_mse / (1 - df_eff/n)^2; df_effs[li] <- df_eff
}
igcv <- which.min(gcv_vals); lambda_gcv <- lam_grid[igcv]
cat("lambda_GCV =", lambda_gcv, " df_eff =", df_effs[igcv], "\n")

```

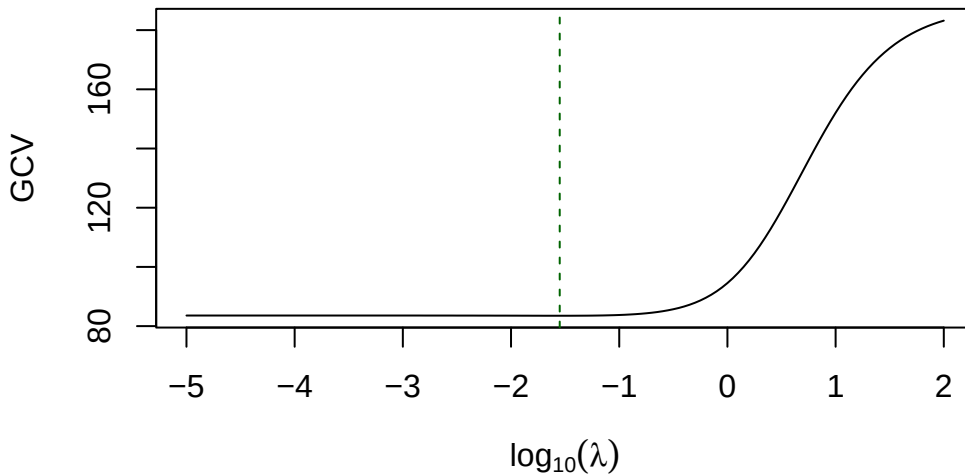
lambda_GCV = 0.02818383 df_eff = 6.688657

```

plot(log10(plot_lam), gcv_vals, type = "l", xlab = expression(log[10](lambda)),
     ylab = "GCV", main = "GCV curve")
abline(v = log10(lambda_gcv), col = "darkgreen", lty = 2)

```

GCV curve



$\lambda_{\text{GCV}} \approx 0.0282$ ($\text{df}_{\text{eff}} \approx 6.69$ of 7).

c.

```
Xtest <- apply_standardize(Xtest_raw, par_all)
evaluate <- function(lam, label) {
  fit <- ridge_fit(Xall, ytrain, lam)
  train_mse <- mean((ytrain - (fit$beta0 + Xall %*% fit$beta))^2)
  test_mse <- mean((ytest - (fit$beta0 + Xtest %*% fit$beta))^2)
  df_eff <- 1 + sum(dvals^2 / (dvals^2 + n*lam))
  data.frame(model = label, lambda = lam, train_MSE = train_mse,
             test_MSE = test_mse, df_eff = df_eff)
}
final_tab <- rbind(evaluate(0, "OLS"), evaluate(lambda_min, "lambda_min"),
                  evaluate(lambda_1se, "lambda_1se"), evaluate(lambda_gcv, "lambda_GCV"))
print(final_tab, row.names = FALSE)
```

	model	lambda	train_MSE	test_MSE	df_eff
	OLS	0.00000000	80.09392	66.32246	7.000000
	lambda_min	0.01995262	80.13543	65.99957	6.773213
	lambda_1se	1.58489319	100.37356	81.19981	3.032870
	lambda_GCV	0.02818383	80.17125	65.90276	6.688657

$\lambda_{\min}$ and λ_{GCV} agree closely ($\text{df}_{\text{eff}} \approx 6.7\text{--}6.8$), fitting almost as flexibly as OLS with a small variance benefit. $\lambda_{1\text{se}}$ picks a much simpler model ($\text{df}_{\text{eff}} \approx 3.0$) that generalizes worse here, but the 1-SE rule optimizes for robustness, not lowest error on one dataset. All three λ 's were fixed before viewing test data; picking whichever minimizes test MSE after the fact would overfit to test-set noise and bias the reported error low.

explain-to-me skill

```
---
name: explain-to-me
description: Explain a homework question in plain language without solving it.
---

# Explain-to-me

Use this skill whenever the user asks you to interpret a homework question.
```

Rules:

- Read only the relevant question and explain what it is asking.
- State the goal, required output, and the main statistical idea.
- Separate the mathematical setup, the tasks, and the deliverables.
- Point the user to the most relevant lecture section.
- Do not compute the final answer or complete the solution.
- If the question is ambiguous, ask one targeted clarifying question before proceeding.
- Keep the explanation concise and student-friendly.

Workflow:

1. Identify the exact question and summarize it in plain language.
2. List the required steps or subparts.
3. Name the relevant concept and notation.
4. Mention a lecture section that helps with context.
5. Offer to clarify definitions or implementation details next.

Relevant lecture references:

- Ridge Regression: Stability Through Shrinkage
- From a Penalized Objective to a Fitted Ridge Model

Use the Week 3 ridge-regression materials as the default reference unless the question is clearly about something else.
Focus on understanding, not solving.